Exploiting Spack for MPICH's CI

Ethan Wong Ken Raffenetti Hui Zhou

Argonne National Laboratory

March 19th, 2026

Speakers

- Ethan Wong
- ewong@anl.gov
- Argonne Leadership Computing Facility

- Ken Raffenetti
- raffenet@anl.gov
- Mathematics and Computer Science

MPICH

MPICH is a leading MPI implementation and a large open-source project.

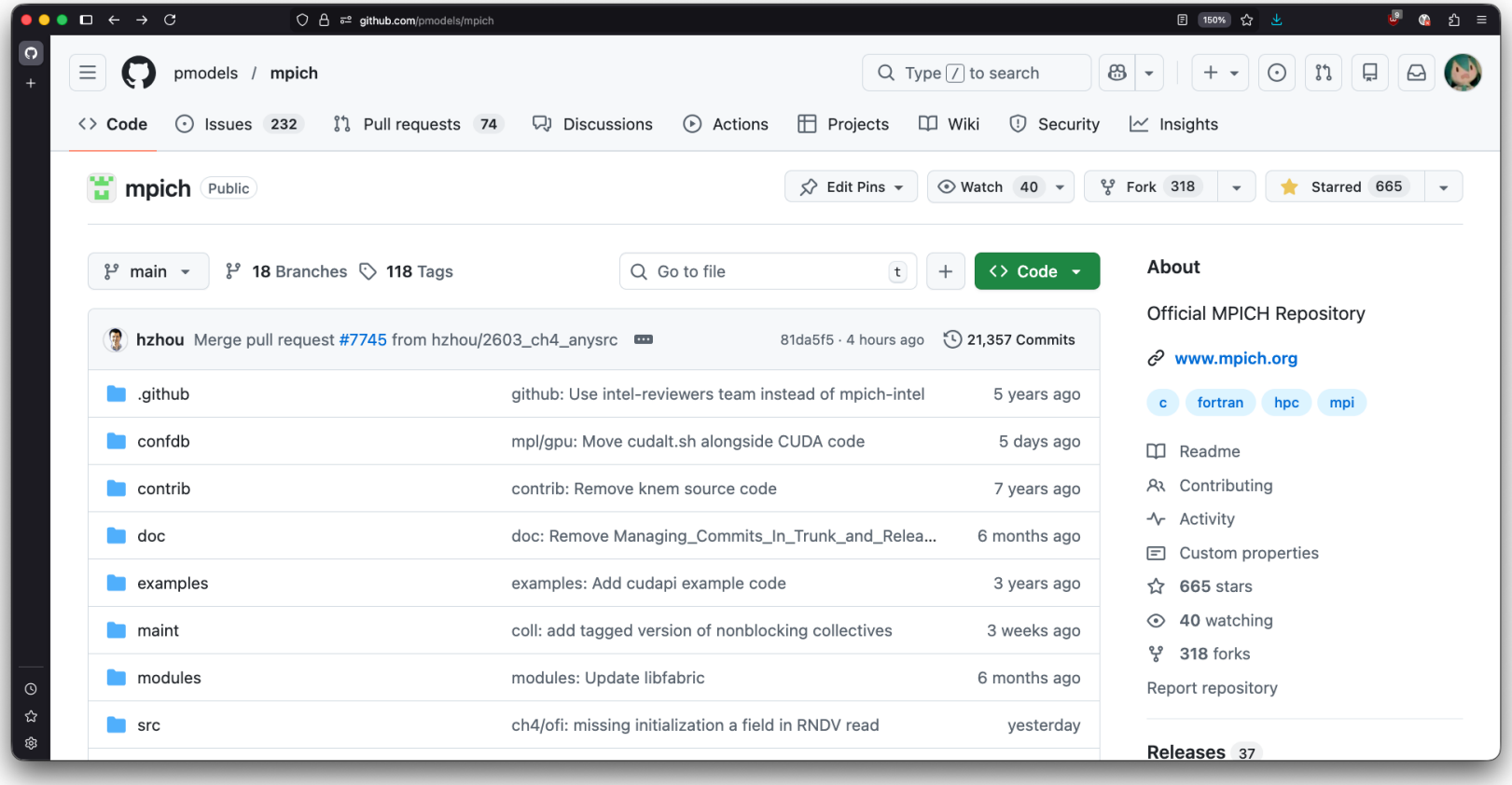


Figure: MPICH GitHub[1]

[1] github.com/pmodels/mpich

Our Maintainer Workflow

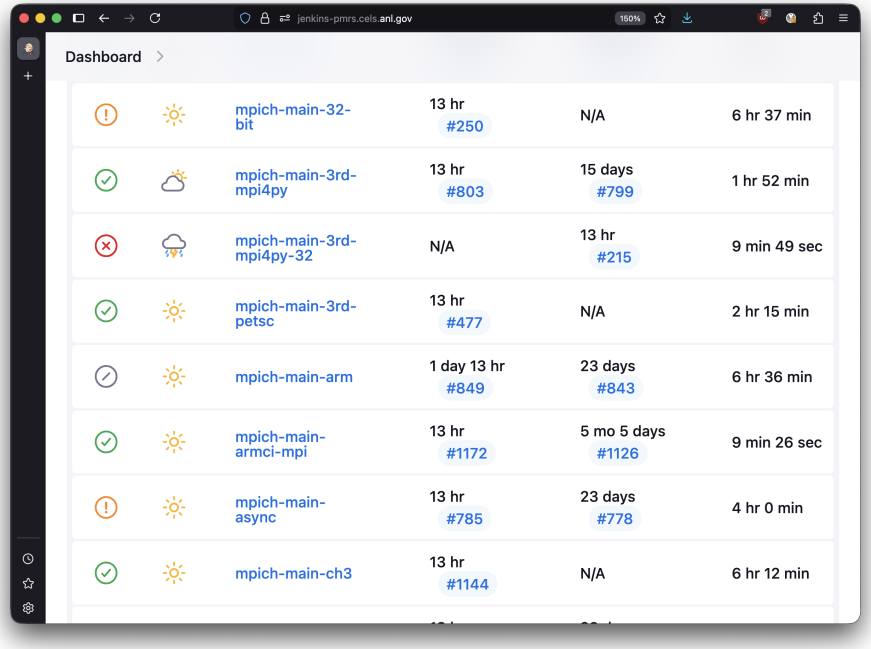


Figure: PMRS Jenkins[a]

[a] `jenkins-pmrs.cels.anl.gov`

CI heavily optimizes our development workflow.

- Holistic coverage
  - Compilers
  - OS
  - `asan`, `debug`, ...
  - Async progress
  - GPU collectives
  - etc...
- All PRs test against CI
- `main` validated nightly

Our Maintainer Workflow

$$\begin{aligned}\text{Nightly Jobs} &\approx 300 \text{ matrix configurations}\\ &\mapsto N \times M \times \ldots \text{ combinations}\\ &\Rightarrow \textbf{1500} \text{ runs } \underline{\text{nightly}}\end{aligned}$$

Our Maintainer Workflow

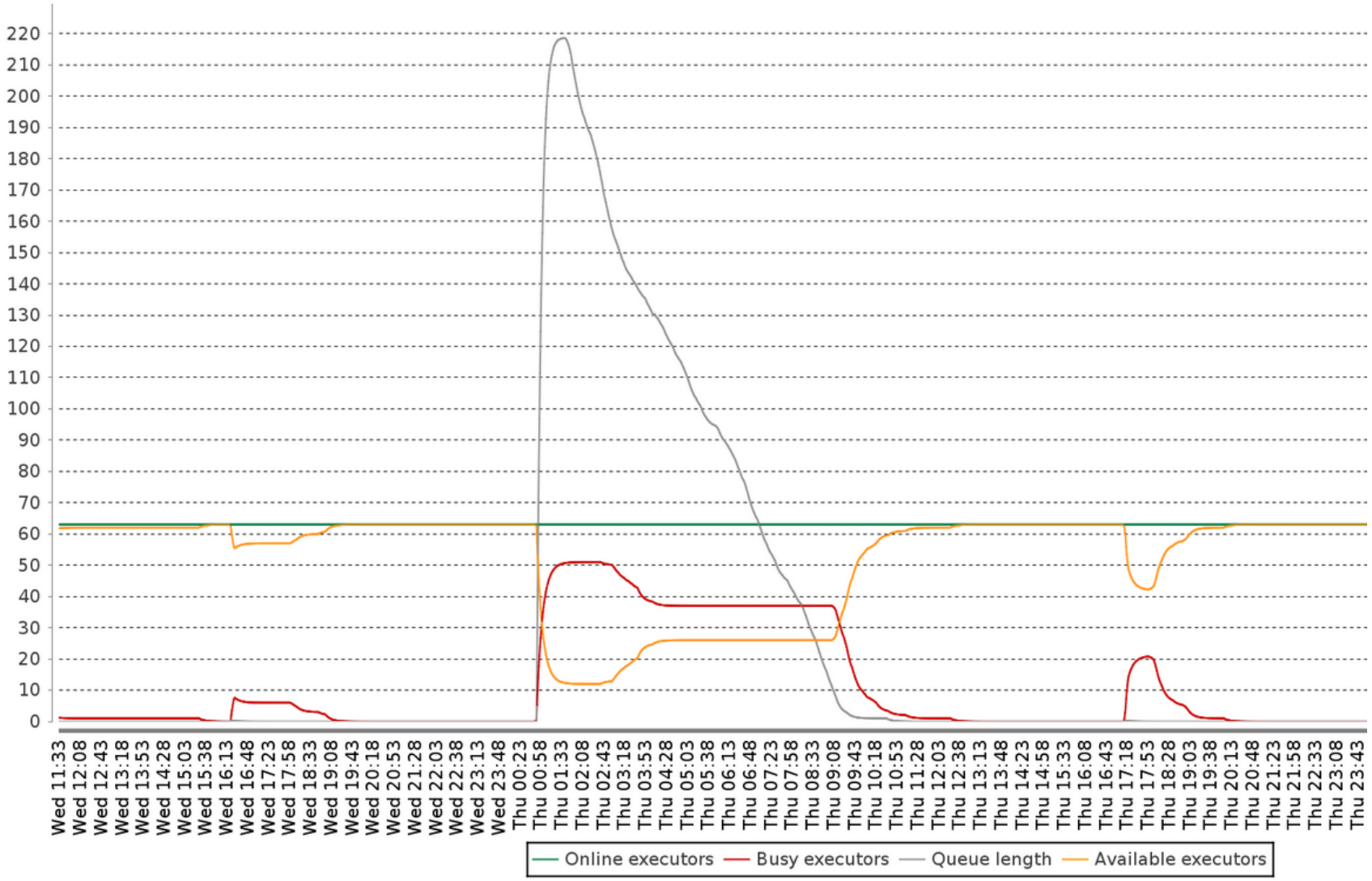


Figure: Jenkins Load Usage Statistics

Qualms

Our CI pipeline is ssslllooooowwwww... :(

Baseline

$\sim$1 hour 30 minutes *per-run*

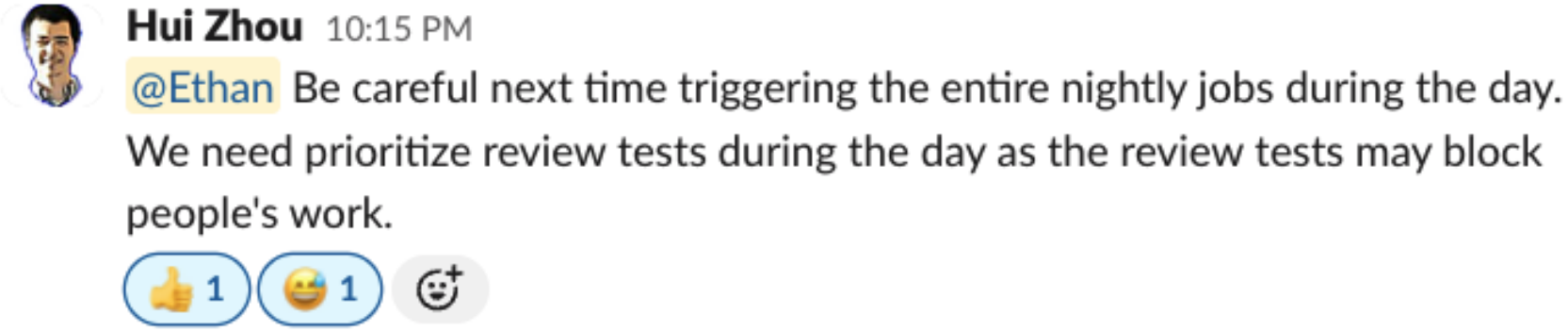
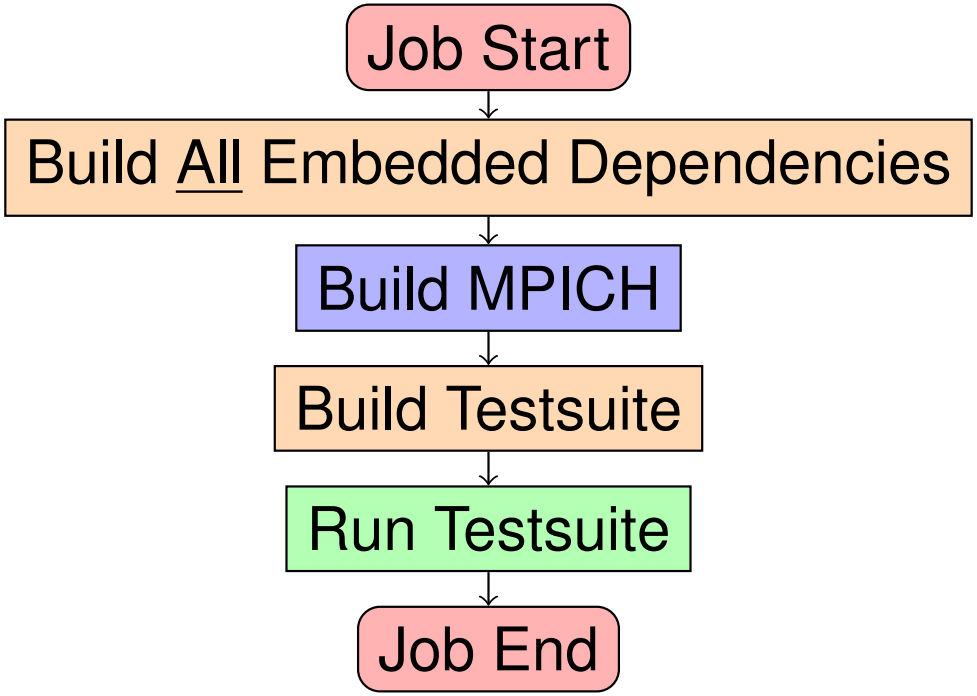


Figure: Re-running Nightly Testsuite at 2 PM

Qualms

Per-Run Rebuilds



- Builds are reliant on host agents' system packages
  - Difficult to maintain consistency
  - Significant effort to test arbitrary versions

$\Longrightarrow$ **inflexible**

What Should We Do?

This seems like a job for a package manager...

Spack?

Spack

1. Aggressive caching of *n* build configurations
2. Declarative build/test environments
   - Explicit *per-package* dependency declaration
3. HPC a first-class citizen

Implementation

How should we do this?

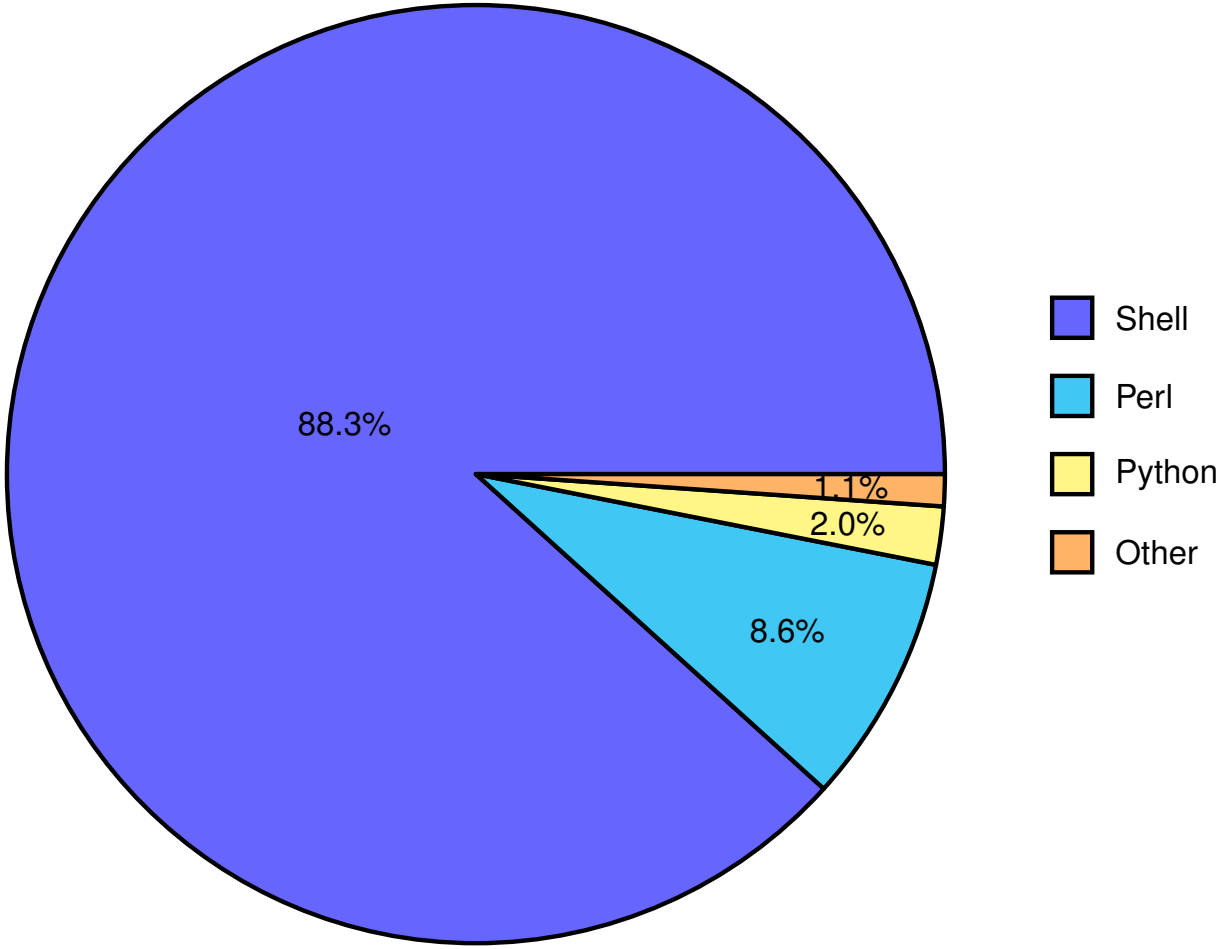


Figure: Jenkins Scripts Repository

Minimal changes $\Longrightarrow$ ***Substitute* compilation with Spack!**

Design

Idea

Let's treat each component of MPICH as a Spack package.

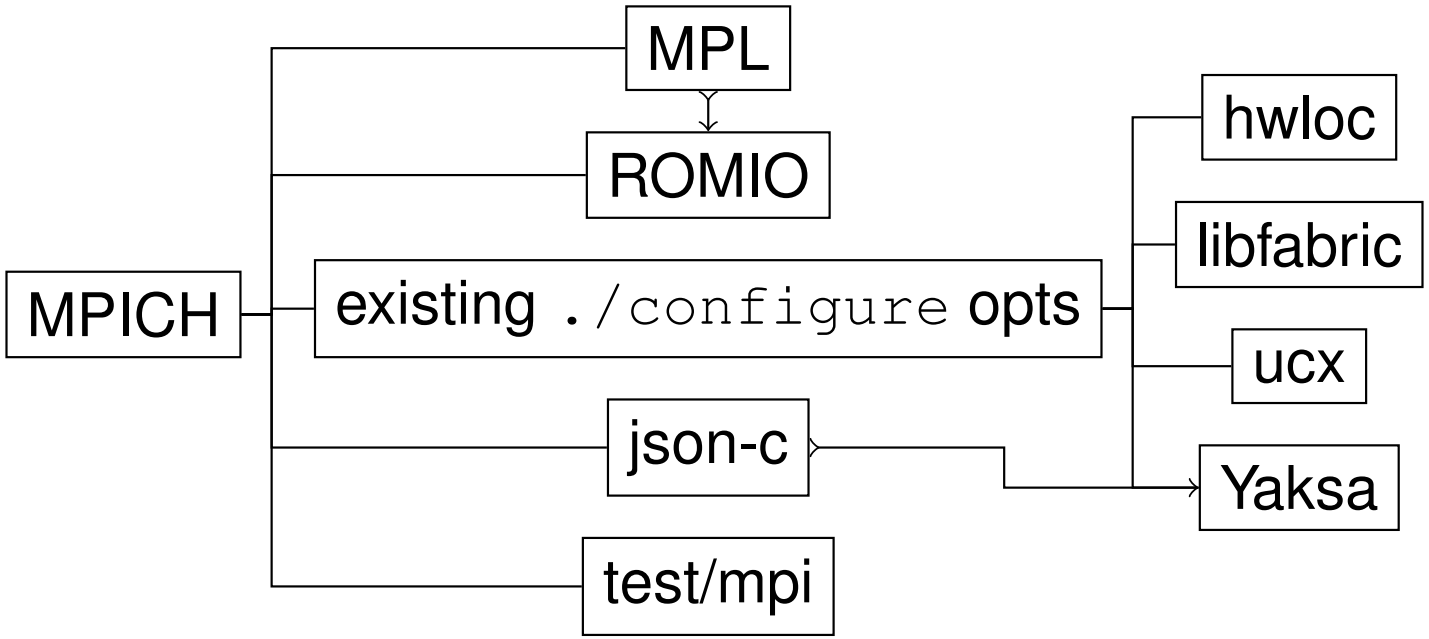


Figure: MPICH Dependency Graph[2]

[2] direct dependents

Design

We can easily define an “Internal PMRS Spack Repository” with each component.

- Direct control of MPICH dependencies[3]
- Can soft-fork upstream package definitions

Examples

```
from spack_repo.builtin.packages.ucx.package import Ucx as BuiltinUcx
from spack.package import *


class Ucx(BuiltinUcx):
        version(
                "1.19.0-embedded",
                git="https://github.com/pmodels/ucx.git",
                commit="4020d2be2271da0ac105d824f26ec76ad533577a",
        )

        @when("@1.19.0-embedded")
        def autoreconf(self, spec, prefix):
                Executable("./autogen.sh")()
```

[3] i.e. forked versions, cherry-picked patches, etc.

Design

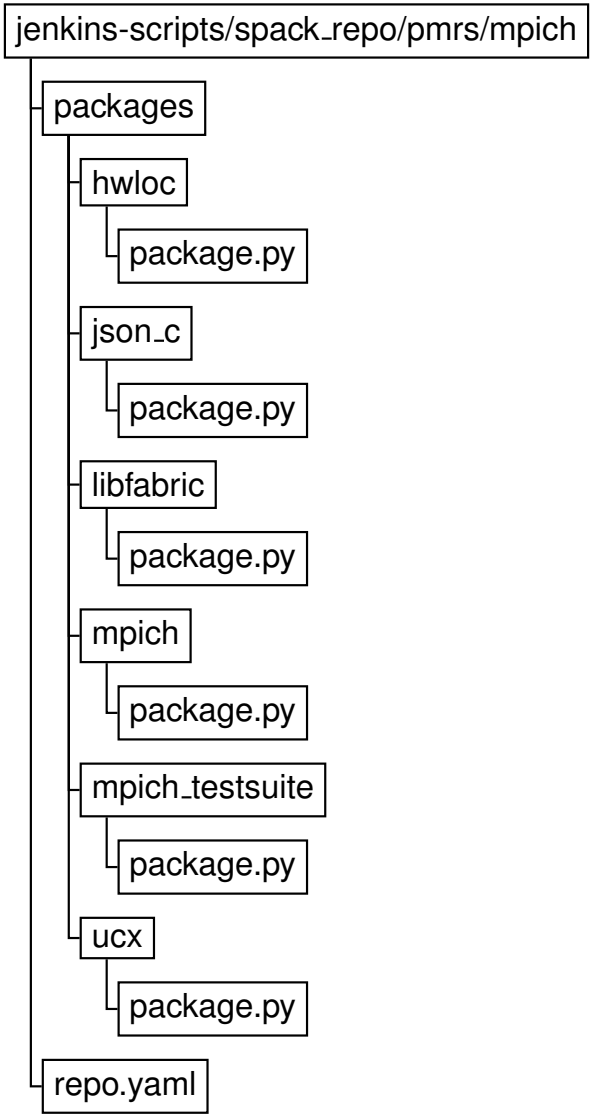


Figure: Internal PMRS Spack Repository

Integration with Jenkins

Our matrix jobs have custom *labels* which correspond to `./configure` options.

Examples

Configuration Matrix	gnu	clang
`ch3-tcp`	✓	✓
`ch3-sock`	✓	✓
`ch4-ofi`	✓	✓

Figure: `mpich-main-osx`[4] CI Job

How do we translate these to `spack install`s?

[4] `jenkins-pmrs.cels.anl.gov/job/mpich-main-osx`

Integration with Jenkins

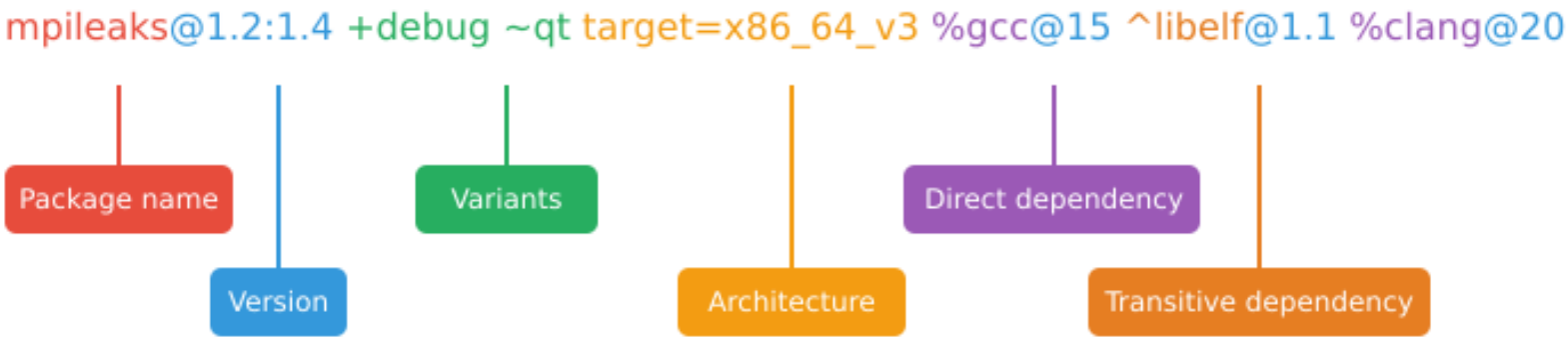


Figure: Spec Syntax[5]

Idea

Let's generate equivalent Spack specs!

- Reuse Label $\mapsto$ `./configure` option mappings
- Variants provide configuration granularity
- Dependencies can be explicitly resolved

[5] `spack.readthedocs.io/en/latest/spec_syntax.html`

Integration with Jenkins

Examples

```
config_opt=' --with-device=ch4:ofi --without-xpmem
--with-wrapper-dl-type=rpath --with-hwloc=embedded
--with-libfabric=embedded'
```

$$\Downarrow$$

```
mpich@develop device=ch4 netmod=ofi ~xpmem
wrapper-dl-type=rpath +hwloc %hwloc@2.9.0-embedded
%libfabric@1.20.1-embedded
```

Idea++

We can generalize this to a tool which templates Spack specs/environments for our CI[6] to concretize and install.

[6] and for us maintainers

PR Jobs

We can also utilize `spack develop` and `@<sha-1>` Git versions to validate contributed PRs *while utilizing Spack*!

Examples

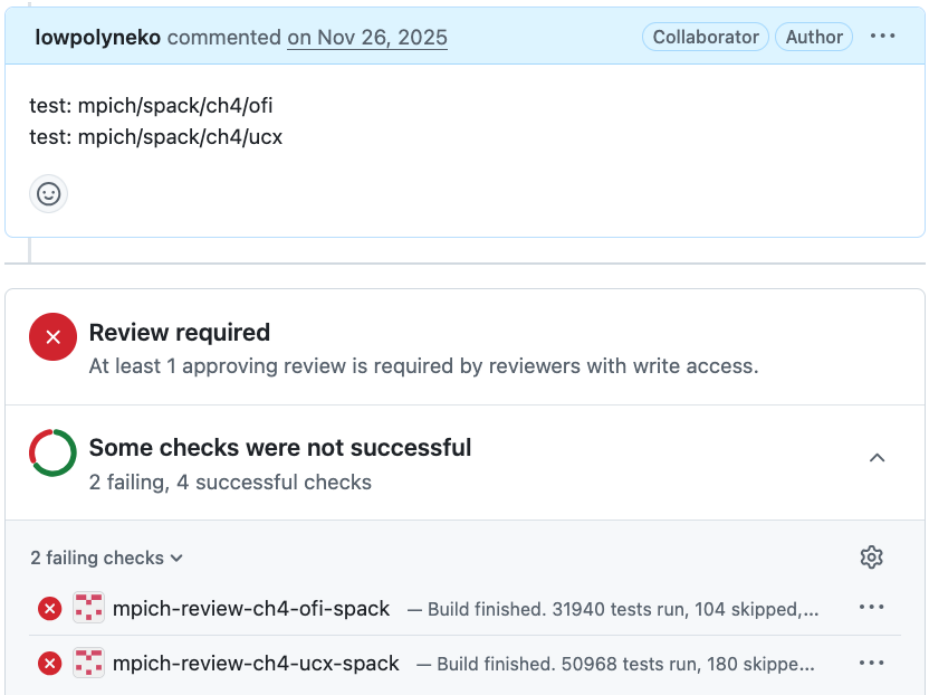


Figure: Example PR Utilizing Spackified CI Jobs[7,8]

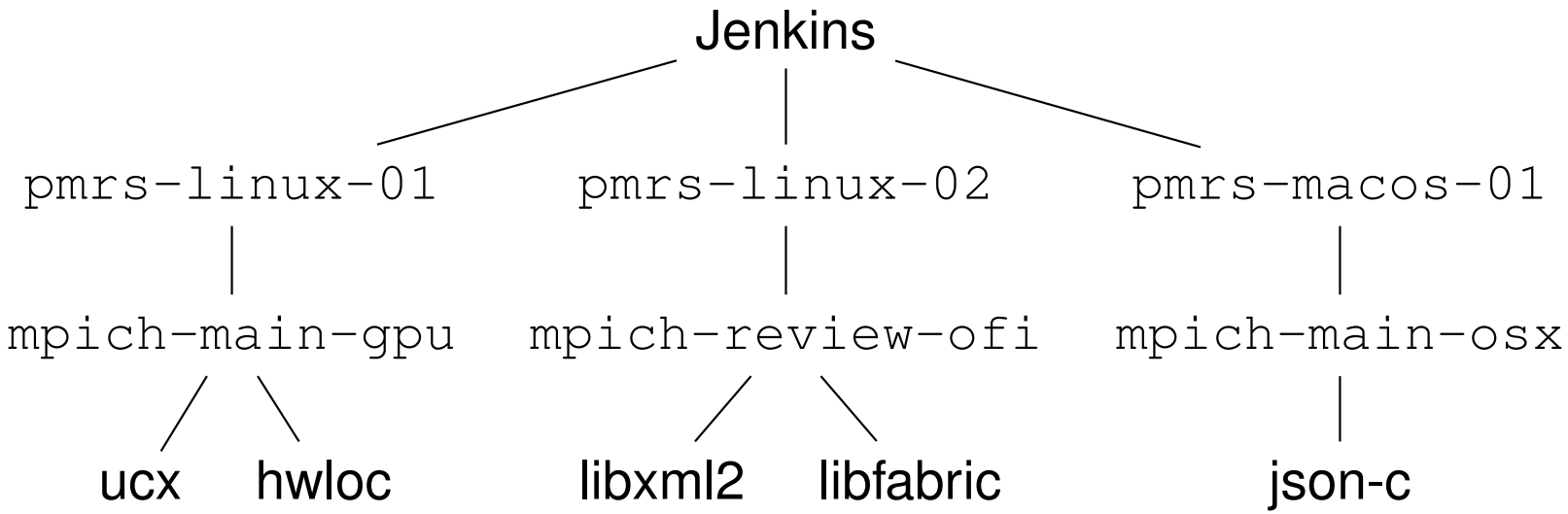
[7] `jenkins-pmrs.cels.anl.gov/job/mpich-review-ch4-ofi-spack`

[8] `jenkins-pmrs.cels.anl.gov/job/mpich-review-ch4-ucx-spack`

Concurrent Jobs

An interesting consequence of integrating Spack into our existing CI infrastructure is that per-job `spack install`s run in parallel.

Examples



Bug!

We ran into a race condition with Spack's installer...

Concurrent Jobs

bugfix: don't stop early when other spacks are installing #51539

Merged haampie merged 1 commit into `develop` from `bugfix-early-stop-on-multi-node-install` on Nov 10, 2025

Conversation 4 | Commits 1 | Checks 30 | Files changed 1

tgamblin commented on Nov 8, 2025 • edited — Member

Fixes #50933.

The installer in #47590 missed a loop condition and wasn't properly waiting on tasks requeued as `BuildStatus.INSTALLING`. This means another Spack is working on that task, and we should wait and not bail early.

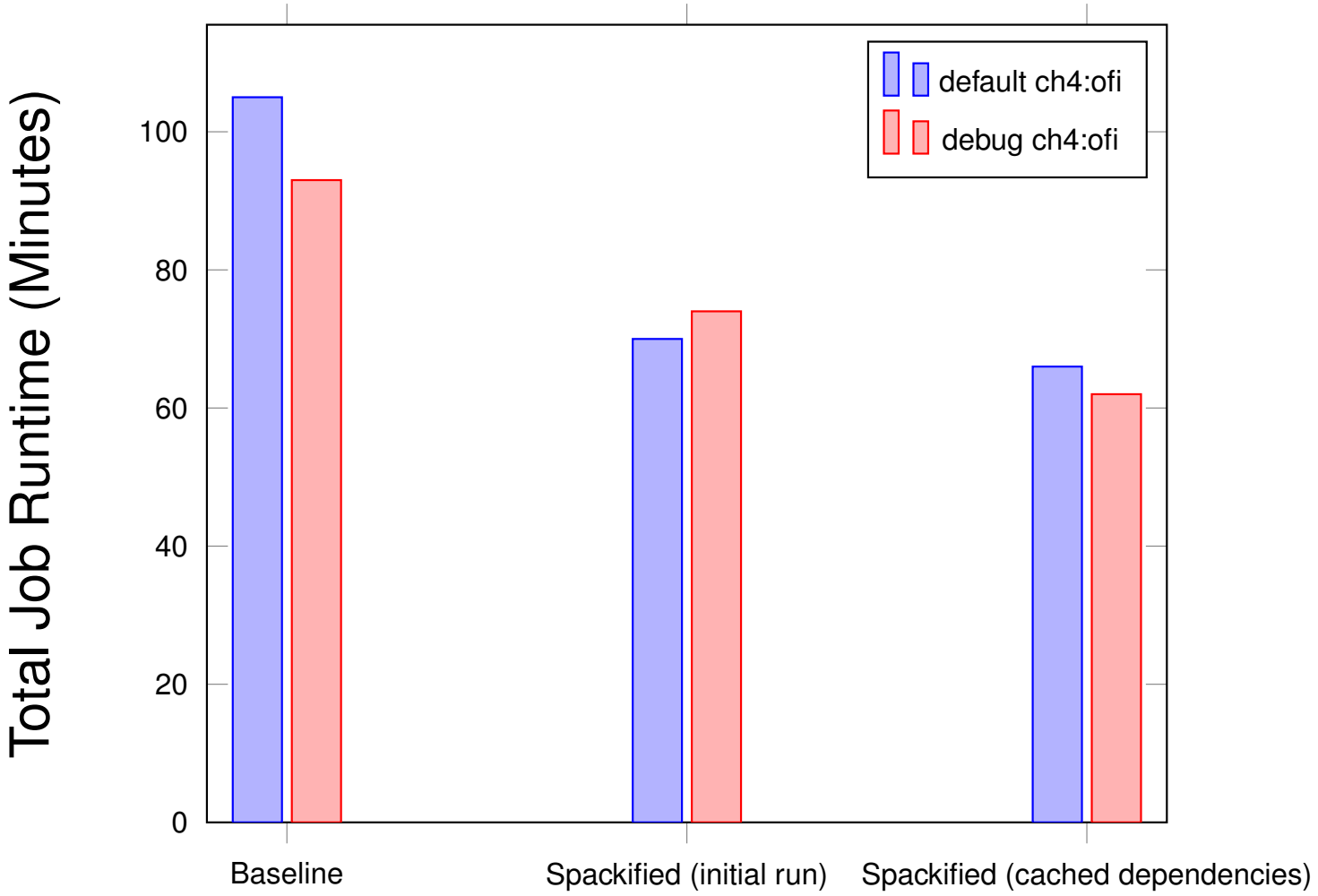
There is also a condition in the installer where we disocver for the first time that a package is installed in cleanup. We need to update the installed dependencies lists of dependents when that happens.

Figure: `github.com/spack/spack/pull/51539`

Thank you, Todd!

Results

On average, our CI builds are now 33% **faster!**



The primary bottleneck now is our single-threaded testsuite.

Results++

As maintainers, we also benefit from massive QoL boosts!

- We can pull any MPICH build from the Spack install cache, prebuilt by our CI
- We have cleanroom[9] environments for our CI jobs
- External dependencies now build independently of MPICH
- For *matrix configurations*, we now cache identical builds

[9] -ish

Future Work

We see huge potential in further integration of Spack for our CI.

Our Ideas

- Refactoring internal, embedded modules to be cachable by Spack
  - MPL
  - ROMIO
  - etc...
- Pre-compilation of our entire testsuite
- Try integrating `ccache` for better compilation granularity
- Mirroring of CI builds for public usage
- Exporting test environment as OCI images?

The future is bright for our CI! ˆwˆ

Recap

Background
- MPICH
- Our Maintainer Workflow

Methodology
- Design/Integration
- Bug!

Results

Closing Remarks
- Future Work

Closing Remarks

Thank you!

- Hopefully this inspires other projects to attempt a similar endeavor.
- *We'll stick around to chat afterwards...*